

OBJECT-ORIENTED DESIGN GUIDE

SOLID Principles

A practical refactoring guide with simple Java examples

Five principles. Five common design problems. Five focused fixes.

Each section explains the failure mode, its effect on the codebase, and the smallest useful refactor.

Why SOLID matters

SOLID is a set of design principles for keeping object-oriented code understandable as it changes. It does not require extra classes everywhere. Its purpose is to reduce the cost and risk of change by giving each class a clear job and by keeping dependencies flexible.

Use this guide like a diagnostic tool: when a class changes for unrelated reasons, rejects valid objects, forces unused methods, or creates its own dependencies, one of the SOLID principles can suggest a smaller and safer design.

The five questions

Single Responsibility: Does this class have one reason to change?

Open/Closed: Can I add a new behavior without editing stable logic?

Liskov Substitution: Can every subtype safely stand in for its base type?

Interface Segregation: Is a client forced to depend on methods it does not need?

Dependency Inversion: Does high-level policy depend on abstractions rather than concrete details?

The examples use Java, but the reasoning applies to any object-oriented language. The goal is not maximum abstraction; it is code that is easier to change, test, and explain.

1. Single Responsibility Principle (SRP)

Definition: A class should have one responsibility and therefore one reason to change.

The problem: Invoice calculates a total, prints a receipt, and saves data. Pricing, presentation, and storage are unrelated concerns.

Why it hurts: A layout change, a database change, or a pricing change can all force edits to the same class. The class becomes harder to test and more likely to break during unrelated work.

Before

```
class Invoice {  
    double total() { return 100; }  
    void print() { System.out.println(total()); }  
    void save() { /* database code */ }  
}
```

After

```
class Invoice { double total() { return 100; } }  
  
class InvoicePrinter {  
    void print(Invoice invoice) { System.out.println(invoice.total()); }  
}  
  
class InvoiceRepository {  
    void save(Invoice invoice) { /* database code */ }  
}
```

How the refactor helps: Each class now changes for one focused reason: business calculation, receipt formatting, or persistence. Tests can target those responsibilities independently.

Practical rule: Split a class when its methods serve different actors or change for unrelated business reasons.

2. Open/Closed Principle (OCP)

Definition: Software entities should be open for extension but closed for modification.

The problem: DiscountService uses an if chain for every customer type. Adding a new discount means editing the same stable method again and again.

Why it hurts: Every new case risks changing existing behavior. The method grows, tests become repetitive, and unrelated discount rules become tightly coupled.

Before

```
class DiscountService {
    double discount(String type, double price) {
        if (type.equals("VIP")) return price * 0.20;
        if (type.equals("STUDENT")) return price * 0.10;
        return 0;
    }
}
```

After

```
interface Discount { double apply(double price); }

class VipDiscount implements Discount {
    public double apply(double price) { return price * 0.20; }
}

class DiscountService {
    double finalPrice(double price, Discount discount) {
        return price - discount.apply(price);
    }
}
```

How the refactor helps: A new rule is added as another Discount implementation. Existing pricing logic stays unchanged, so the new behavior is isolated and easier to test.

Practical rule: When behavior varies by type or policy, introduce a small strategy interface instead of extending a conditional chain.

3. Liskov Substitution Principle (LSP)

Definition: Objects of a subtype must be usable wherever their base type is expected without changing correct program behavior.

The problem: Penguin extends Bird, but Bird promises fly(). Penguin cannot honor that promise and throws an exception.

Why it hurts: Code that accepts Bird can fail at runtime for a valid subtype. Inheritance stops representing a safe is-a relationship.

Before

```
class Bird { void fly() { } }

class Penguin extends Bird {
    void fly() { throw new UnsupportedOperationException(); }
}

void makeFly(Bird bird) { bird.fly(); }
```

After

```
interface Bird { void eat(); }
interface FlyingBird extends Bird { void fly(); }

class Sparrow implements FlyingBird {
    public void eat() { }
    public void fly() { }
}

class Penguin implements Bird { public void eat() { } }
```

How the refactor helps: The capability to fly is modeled separately. A method that needs flying birds accepts FlyingBird, so no valid object is forced to reject the contract.

Practical rule: Do not model a subtype with inherited operations it cannot perform correctly. Extract the real capability instead.

4. Interface Segregation Principle (ISP)

Definition: Clients should not be forced to depend on methods they do not use.

The problem: A basic printer is required to implement `scan()` and `fax()` only because one large `Machine` interface contains every feature.

Why it hurts: Unsupported operations appear, callers see misleading capabilities, and simple implementations become coupled to features they do not provide.

Before

```
interface Machine {
    void print();
    void scan();
    void fax();
}

class BasicPrinter implements Machine {
    public void print() { }
    public void scan() { throw new UnsupportedOperationException(); }
    public void fax() { throw new UnsupportedOperationException(); }
}
```

After

```
interface Printer { void print(); }
interface Scanner { void scan(); }
interface Fax { void fax(); }

class BasicPrinter implements Printer {
    public void print() { }
}
```

How the refactor helps: Each client depends only on the capability it actually needs. A multifunction device can implement several focused interfaces, while a basic printer stays honest and simple.

Practical rule: Prefer several small role-based interfaces over one broad interface designed for every possible client.

5. Dependency Inversion Principle (DIP)

Definition: High-level policy should depend on abstractions, not directly on low-level implementation details.

The problem: NotificationService creates EmailSender itself. The business rule is now tightly coupled to email and cannot be reused for SMS, push notifications, or tests.

Why it hurts: Changing the delivery method requires editing policy code. Unit tests need real infrastructure or complicated setup instead of a simple fake dependency.

Before

```
class EmailSender {
    void send(String message) { /* send email */ }
}

class NotificationService {
    private final EmailSender sender = new EmailSender();
    void notifyUser(String message) { sender.send(message); }
}
```

After

```
interface MessageSender { void send(String message); }

class EmailSender implements MessageSender {
    public void send(String message) { /* send email */ }
}

class NotificationService {
    private final MessageSender sender;
    NotificationService(MessageSender sender) { this.sender = sender; }
    void notifyUser(String message) { sender.send(message); }
}
```

How the refactor helps: NotificationService depends on the behavior it needs, not on one transport. The application can inject email, SMS, or a fake sender for a fast unit test.

Practical rule: Pass important collaborators into a class through constructors and depend on the smallest useful abstraction.

A quick refactoring checklist

SRP: If a class changes because of database, user interface, and business-rule changes, separate those responsibilities.

OCP: If every new case adds another if or switch branch in stable code, move the varying behavior behind a strategy or policy.

LSP: If a subtype throws unsupported-operation errors or weakens assumptions, redesign the hierarchy around true capabilities.

ISP: If an implementation contains empty methods or unsupported features, split the interface into smaller roles.

DIP: If a service creates its own database, API, or sender object, inject an abstraction instead.

SOLID works best as a response to a real design smell. Start small, preserve working behavior, and refactor only enough to make the next change safer.